

Paidly — Payroll, Payslip & Leave Structure

Workforce Core reference. One employee identity. Payroll calculates pay. Leave feeds unpaid days into that calculation. A payslip is the locked statement issued from a finalized pay run.

Timezone for all calculations: Africa/Johannesburg . Statutory rates live in the database, not in React.

ONE WORKFORCE CORE

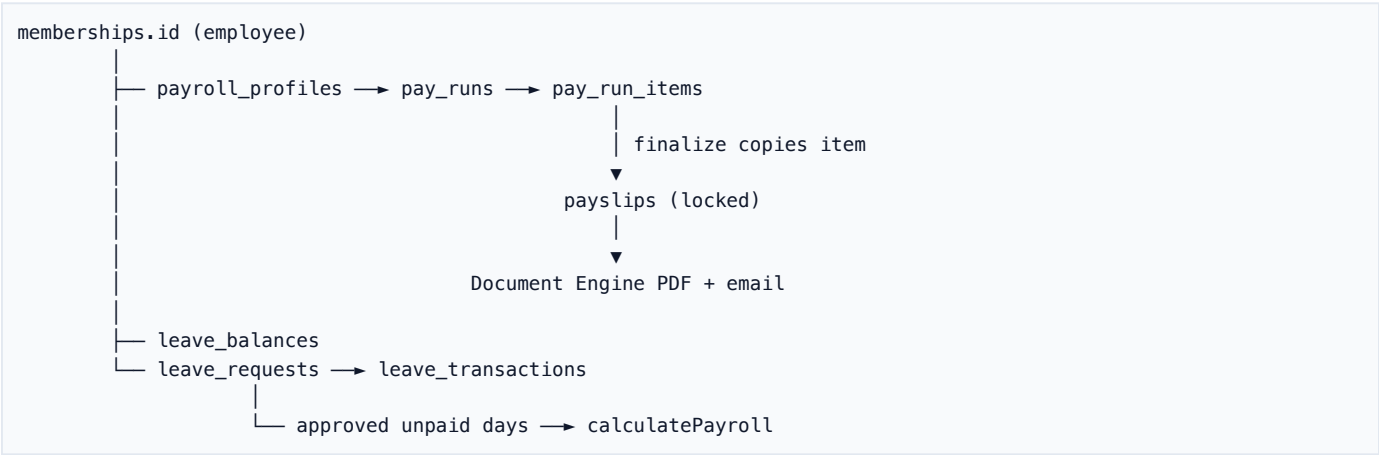
These three features are **one system**, not three products. Employee identity is `memberships.id` . Do not create a parallel `employees` table. UI labels such as `Name (EMP-002)` are display-only; foreign keys store the membership UUID.

organizations	
→ memberships.id	(employee)
→ payroll_profiles	
→ pay_runs → pay_run_items → payslips.membership_id	
→ leave_types / leave_balances / leave_requests / leave_transactions	
→ attendance_profiles	(1:1 stub)
→ company_invites.membership_id	(portal)

Concept	Persistence
Employee	memberships.id
Payslip	payslips (issued statement; generated from a pay run)
Payroll run	pay_runs + pay_run_items + payroll_profiles
Leave ledger	leave_types , leave_balances , leave_transactions , leave_requests

`payslips.employee_id` remains the printed employee number. The UUID link is `payslips.membership_id` .

HOW THEY CONNECT



Unpaid approved leave overlapping the pay period becomes a **negative UNPAID earning**. Paid leave does not change salary.

1. PAYROLL

Job: Calculate and lock a period’s pay.

Tables

Table	Role
payroll_profiles	Salary, pay type, tax IDs — derived from membership
payroll_component_types	Recurring earnings / deductions (allowance, pension, ...)
payroll_statutory_rules	Versioned PAYE / UIF / etc. (platform + org override)
pay_runs	One period batch

Table	Role
pay_run_items	Per-employee snapshot (salary + unpaid-leave impact)

Math — only in one place

shared/payroll/calculatePayroll.js is the only payroll formula. Pure: no I/O, no browser clock.

```
basic → + earnings → - UNPAID → + overtime = gross
gross → statutory (PAYE, UIF, ...) + other deductions = net
```

- **Basic** from `monthly_salary / hourly × hours / daily × days`
- **Unpaid** = `basic × (unpaid_days / working_days_in_period)` via `shared/payroll/unpaidLeaveImpact.js`
- **Statutory rules:** `percent, capped_percent, fixed, tax_brackets`
- Org rules override platform rules for the same code in the period window
- Pay-run items snapshot `base_salary` and unpaid-leave impact so later salary changes do not rewrite history

Pay-run lifecycle

```
draft → processing → calculated → awaiting_approval → approved → (finalize) → paid
                                                    ↘ cancelled
```

Locked statuses (`approved` , `paid`) cannot be silently rewritten. Corrections use an **adjustment pay run**.

Step	What happens
Create	Open run for a period; <code>syncPayRunEmployees</code> adds new hires
Calculate	Loads profiles, recurring components, statutory rules, approved unpaid leave → writes item snapshots
Submit / approve	Separation of duties: a different payroll admin must approve if more than one exists
Finalize	Locked <code>payslips</code> rows copied from items; emails sent
Paid	Run + payslips marked paid
Recalc after finalize	Blocked. Create an adjustment run

Server API

`/api/payroll/*` rewrites onto `api/company` (Hobby function ceiling — no new serverless file).

Route	Action
GET overview	Payroll dashboard
POST preview	Dry-run <code>calculatePayroll</code>
GET/POST profiles	Sync / upsert payroll profiles
GET/POST runs	List / create pay runs
POST runs/:id/calculate	Calculate items
POST runs/:id/submit	Send for approval
POST runs/:id/approve	Approve
POST runs/:id/finalize	Generate locked payslips + email
POST runs/:id/paid	Mark paid
POST runs/:id/send	Resend payslip emails
GET/POST statutory	List / upsert statutory rules
GET me	Employee self-service payslips

Code: `server/src/payroll/payrollRoutes.js` → `payrollService.js`. Client: `src/pages/Payroll.jsx`, `src/services/PayrollApiService.js`.

2. PAYSZIP

Job: Issued pay statement. Not the employee source of truth. Not a Documents Hub row.

Do not dual-write payslips into `documents`. The Documents Hub is not a second store for commercial documents.

Canonical issue path (preferred)

```
approved pay run
  → finalizePayRun
  → insert payslips (locked: true)
  → link pay_run_items.payslip_id
  → send email with /PublicPayslip?token=
```

The row copies the pay-run item: `basic`, `overtime`, `allowances`, `statutory`, `net`, `calculation_breakdown`, `leave_summary`.

Email is a secure `/PublicPayslip?token=` link — **never** an unencrypted payroll PDF attachment and never a `/view/` public invoice URL.

Secondary path

Standalone compose (`CreatePayslip`) still exists for out-of-band drafts and **must** attach `membership_id`. It is not the normal issue path.

Document Engine (delivery only)

Same compose / send / PDF stack as invoices and quotes, **different table** (`payslips`):

- PDF: `PayslipDocument` / `src/document-engine/pdf/payslip.jsx`
- Events: `created` · `sent` · `delivered` · `opened` · `clicked` (`view_payslip`) · `downloaded`
- Payslips must **never** record `paid`, `viewed_not_paid`, `due_soon`, `due_today`, `overdue`, `payment_intent`, `accepted`, or `rejected`
- Event payload must not store `net_pay`, tax amounts, bank details, or ID numbers
- Public view: token + email gate (`api/public-share` + `api/_publicPayslipShared.js`)
- Self-service match: `employee_user_id` **or** `membership_id` **or** employee email

Locked payslips cannot be silently rewritten.

3. LEAVE

Job: Working-day ledger + approval. Hub `leave_request` documents are leftovers — they are not this ledger.

New applications go through `/api/leave`. HR leave (balances, accrual, approval) is canonical on `leave_requests` / `leave_balances` / `leave_transactions`.

Tables

Table	Role
<code>leave_types</code>	Policy: ANNUAL, SICK, FAMILY, UNPAID, STUDY, MATERNITY, PARENTAL
<code>leave_balances</code>	Per employee × type × year: entitled, accrued, used, pending
<code>leave_requests</code>	Application + status
<code>leave_transactions</code>	Ledger (opening, accrual, hold, release, approved, adjustment)
<code>leave_approval_tokens</code>	Hashed single-use email decide links

Available = `accrued` – `used` – `pending` (`computeLeaveBalance` in `shared/leave/leaveMath.js`).

Entitled is policy allocation for the leave year — not a mutation target.

Request flow

```
preview / apply
  → validateLeaveApplication (working days, overlap, balance)
  → status = pending
  → paid types: pending_hold on the ledger
  → email manager (or HR) via token
    ↳ approve → pending_release + approved_leave (used ↑)
    ↳ reject / cancel → pending_release only
```

- Working days counted server-side (`countWorkingDays`; weekends excluded by default)

- Unpaid types skip the balance check and do not hold days
- Approver: assigned manager, or HR override
- Token page (/api/public-leave → api/public-share) cannot see payroll or other employees
- If unpaid leave is approved **after** a finalized run, a workforce event flags needs_adjustment_run

Default leave types (seeded per org)

Code	Paid	Accrual
ANNUAL	yes	monthly, 21 days
SICK	yes	annual, 10
FAMILY	yes	annual, 3
UNPAID	no	none — this is the payroll hook
STUDY	yes	annual, 5
MATERNITY	no	none
PARENTAL	yes	annual, 10

Leave request statuses

draft → pending → approved | rejected | cancelled

Transaction kinds

opening · accrual · approved_leave · pending_hold · pending_release · adjustment · reversal

Server API

/api/leave/* also rewrites onto api/company .

Route	Action
GET/POST types	List / upsert leave types
GET me	Own balances + requests
POST preview	Working days + remaining balance
POST apply	Submit request (holds pending days)
GET requests	Team / HR list (scoped)
POST requests/:id/approve	Approve
POST requests/:id/reject	Reject (reason required)
POST requests/:id/cancel	Cancel own (or HR override)
POST adjust	Manual balance adjustment
GET calendar	Team calendar
GET employees	Employees in leave scope

Code: server/src/leave/leaveRoutes.js → leaveService.js . Shared: shared/leave/leaveMath.js , validateLeave.js . Client: src/pages/Leave.jsx , LeaveCalendar.jsx , MyPayroll.jsx .

Public decide: api/_publicLeaveShared.js via hashed token. The token page is scoped to that one request.

SURFACES (SAME ROWS, DIFFERENT ROLES)

Who	Pages	Sees
Employee	/MyPayroll , apply leave	Own payslips + own leave
Manager	/ManagerPortal	Direct reports only, no salary
HR	/Employees , /Leave , /Payroll	Admin + leave
Finance	/Payroll	Pay runs (manager + job_function=finance)
Anyone with a token	/PublicPayslip , /PublicLeaveApproval	That token only

RBAC uses existing company permissions:

- Manager + `job_function=hr` → employee / leave admin
- Manager + `finance` → payroll
- POS-only staff cannot administer payroll or workforce
- Compensation is omitted from `/api/company/employees` unless `manage_payroll` or the row is the caller

Settings → Team stays invites and roles. It is not a second employee directory.

PROVISIONING

`provisionEmployeeWorkforce()` is idempotent. On `employee.created` it auto-creates:

- payroll profile
- leave balances (year-to-date accrual)
- attendance profile
- `workforce_notification_prefs`
- audit row

Invite accept (`accept_company_invite_token`) links `company_invites.membership_id` onto the pre-created membership (`user_id` was null) and emits `employee.portal.activated` . POS till invites stay on the existing pass path and do not use that link.

Open pay runs pick up new hires via `syncPayRunEmployees` (calculate + refresh).

CODE MAP

Layer	Payroll	Leave	Payslip
Shared math	<code>shared/payroll/calculatePayroll.js</code>	<code>shared/leave/leaveMath.js</code>	<code>shared/payroll/payslipNumber.js</code>
Constants	<code>shared/payroll/constants.js</code>	same file (leave statuses)	—
Service	<code>server/src/payroll/payrollService.js</code>	<code>server/src/leave/leaveService.js</code>	finalize + Document Engine
Routes	<code>payrollRoutes.js</code>	<code>leaveRoutes.js</code>	public-share + compose pages
Authz	<code>payrollGate.js</code>	<code>leaveAuthz.js</code>	self-service match on <code>payslips</code>
UI	<code>Payroll.jsx</code>	<code>Leave.jsx</code> , <code>LeaveCalendar.jsx</code>	<code>Payslips.jsx</code> , <code>PayslipDocument.jsx</code>
Employee	<code>MyPayroll.jsx</code>	apply from <code>MyPayroll</code> / <code>Leave</code>	public token viewer

RULES THAT KEEP THIS FROM FORKING

1. **One employee ID:** `memberships.id` . Do not add an `employees` table.
2. **One payroll formula:** `calculatePayroll` . Statutory rates in `payroll_statutory_rules` .
3. **Payslips are output** of a pay run, not a second payroll system.
4. **HR leave lives on the leave ledger**, not Documents Hub.
5. **Do not mark paid from a browser callback**, and do not build a second payment stack for payroll.
6. Finalized pay runs and locked payslips cannot be silently rewritten. Corrections use an adjustment pay run.

A new payable module needs a `source_kind` + settlement adapter on the Payment Engine. It does not need its own payment system. Payroll / payslips / leave is the failure mode that rule exists to prevent.